HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA ĐÀO TẠO SAU ĐẠI HỌC

BÁO CÁO BÀI TẬP 2

Môn học: Nhận dạng mẫu

Cài đặt và đánh giá Logistic Regression bằng NumPy, so sánh với scikit-learn trên bộ dữ liệu Rice (Cammeo và Osmancik)

Giảng viên: **TS. Nguyễn Xuân Đức**

Học viên: **Trần Vũ Trường**

Mã học viên: **B26CHKH096**

Lớp: **M26CQKH02-B**

Hà Nội, tháng 10 năm 2026

Tóm tắt

Báo cáo cài đặt Logistic Regression nhị phân bằng NumPy (score, sigmoid, log loss, gradient descent cập nhật trọng số và bias) và so sánh với `sklearn.linear_model.LogisticRegression` trên bộ dữ liệu Rice (3.810 hạt gạo, 7 đặc trưng hình thái, 2 giống). Hai bản được cho tối ưu cùng một hàm mục tiêu nên cho kết quả trùng nhau trên tập Test: accuracy 0,9239, F1 0,9127, ROC-AUC 0,9814, log loss 0,1767; xác suất dự đoán lệch nhau tối đa $3{,}0 \times 10^{-5}$. Khác biệt nằm ở tốc độ: gradient descent cần 15.476 vòng lặp, bộ giải lbfgs của sklearn cần 43 vòng, phương pháp Newton tự cài cần 6 vòng. Nguyên nhân là các đặc trưng kích thước gần cộng tuyến. Lỗi phân loại tập trung ở các hạt có kích thước trung gian giữa hai giống.

1. Bài toán và mục tiêu

Đối tượng: một hạt gạo được chụp ảnh. **Đầu vào:** 7 đặc trưng hình thái trích từ ảnh. **Đầu ra:** giống gạo, Cammeo hoặc Osmancik. Quy ước lớp dương y = 1 là Cammeo; precision, recall và F1 trong báo cáo đều tính cho lớp này.

Mục tiêu: (1) tự cài Logistic Regression bằng NumPy, không dùng mô hình hay bộ tối ưu có sẵn; (2) huấn luyện `LogisticRegression` của scikit-learn trên cùng dữ liệu đã tiền xử lý; (3) so sánh hai bản bằng accuracy, precision, recall, F1, ROC-AUC, log loss và ma trận nhầm lẫn; (4) nhận xét về sự hội tụ, chênh lệch giữa hai bản và các lỗi phân loại thường gặp.

2. Dữ liệu

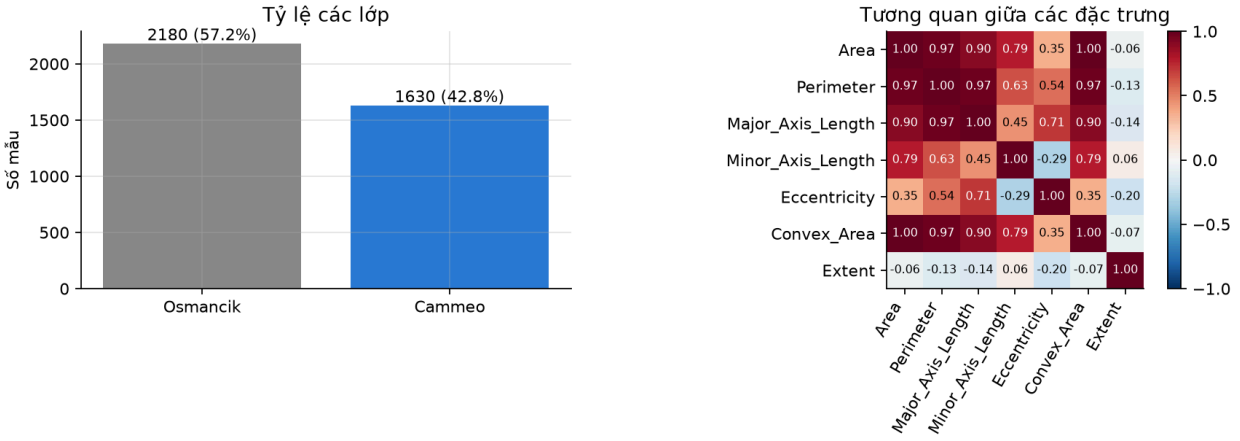
Bộ dữ liệu Rice (Cammeo and Osmancik) lấy từ UCI Machine Learning Repository (id 545), do Cinar và Koklu công bố năm 2019. Dữ liệu gồm 3.810 mẫu, không có ô thiếu và không có dòng trùng lặp.

Đặc trưng	Ý nghĩa
Area	Số pixel bên trong đường biên hạt gạo
Perimeter	Chu vi đường biên
Major_Axis_Length	Độ dài trục chính (đoạn dài nhất vẽ được trên hạt)
Minor_Axis_Length	Độ dài trục phụ
Eccentricity	Độ lệch tâm của elip tương đương (gần 1 là dẹt dài)
Convex_Area	Số pixel của bao lồi nhỏ nhất chứa hạt
Extent	Tỷ lệ diện tích hạt trên diện tích hình chữ nhật bao quanh

Bảng 1. Bảy đặc trưng của bộ dữ liệu Rice.

Nhãn và tỷ lệ lớp: Osmancik 2.180 mẫu (57,2%), Cammeo 1.630 mẫu (42,8%). Hai lớp lệch nhẹ; mô hình luôn đoán lớp đông hơn sẽ đạt accuracy khoảng 57%, nên accuracy vẫn dùng được nhưng cần đọc kèm precision và recall.

Cộng tuyến: các đặc trưng kích thước tương quan rất mạnh với nhau (Area và Convex_Area: 0,999; Area và Perimeter: 0,966). Đặc điểm này không làm giảm độ chính xác nhưng quyết định tốc độ hội tụ của gradient descent (mục 6.1).



Hình 1. Tỷ lệ hai lớp và ma trận tương quan giữa các đặc trưng.

3. Biểu diễn và tiền xử lý

- **Chia dữ liệu:** phân tầng theo nhãn thành Train 2.286 mẫu (60%), Validation 762 mẫu (20%) và Test 762 mẫu (20%), hạt giống ngẫu nhiên cố định. Train dùng để ước lượng tham số, Validation để chọn ngưỡng, Test chỉ dùng cho báo cáo cuối (Slide 1, tr. 19).
- **Chuẩn hoá:** $z = (x - \mu) / \sigma$, với μ và σ học trên Train rồi dùng lại cho Validation và Test (Slide 5, tr. 21). Làm vậy để không rò rỉ thông tin từ Test vào bước tiền xử lý (Slide 3, tr. 14).
- **Cả hai bản dùng chung** ma trận X_{train} đã chuẩn hoá này.

4. Phương pháp

4.1. Cài đặt bằng NumPy

Mã nguồn nằm trong `src/logreg_numpy.py`, chỉ dùng phép toán mảng. Ký hiệu theo Slide 6 (tr. 10–22):

$$s = w^T x + b, \quad p = 1 / (1 + e^{-s})$$

$$L = (1/n)\, \Sigma_i - [\, y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \,], \quad J = L + \lambda\, \Sigma_j\, w_j^2$$

$$\nabla_w J = X^T (p - y) / n + 2\lambda w, \quad \partial J / \partial b = mean(p - y)$$

$$w \leftarrow w - \eta\, \nabla_w J, \quad b \leftarrow b - \eta\, \partial J/\partial b$$

- **Ổn định số:** sigmoid tách hai nhánh $s \geq 0$ và $s < 0$ để không gọi hàm mũ với số dương lớn; log loss tính trực tiếp từ score bằng $\ln(1 + e^s) - y \cdot s$ nên không gặp $\ln(0)$.
- **Regularization:** sklearn tối thiểu hoá $\frac{1}{2}\|w\|^2 + C \sum \ell_i$. Chia cho $C \cdot n$ được đúng dạng J ở trên với $\lambda = 1/(2Cn)$. Với $C = 1$ và $n = 2.286$, $\lambda = 2{,}2 \times 10^{-4}$. Bias không bị phạt. C điều khiển regularization, không phải bước học (Slide 6, tr. 21–22).
- **Learning rate:** $\eta = 1/L_{max} = 0{,}873$, trong đó $L_{max} = \lambda_{max}(\tilde{X}^T\tilde{X})/(4n) + 2\lambda = 1{,}146$ là cận trên độ cong của J (do $p(1 - p) \leq 1/4$). Với bước này J chắc chắn giảm ở mọi vòng lặp.
- **Khởi tạo:** $w = 0$, $b = 0$. **Cập nhật đồng thời** w và b từ cùng một gradient (Slide 5, tr. 11–12).
- **Điều kiện dừng:** $\|\nabla J\|_2 < 10^{-6}$, hoặc tối đa 200.000 vòng lặp.

4.2. Kiểm thử cài đặt

- **Ví dụ tính tay trong slide** (Slide 6, tr. 14 và tr. 21): với $x = (1; 2)$, $w = (0{,}8; -0{,}3)$, $b = -1$, $y = 1$, $\eta = 0{,}1$, mã nguồn cho $s = -0{,}8$; $p = 0{,}310$; $\ell = 1{,}17$; sau một bước $w = (0{,}869; -0{,}162)$, $b = -0{,}931$, $p = 0{,}405$, $\ell = 0{,}90$. Khớp với slide.
- **Kiểm tra gradient:** sai số tương đối giữa gradient giải tích và sai phân hữu hạn trung tâm là $7{,}9 \times 10^{-11}$.

4.3. Scikit-learn

`LogisticRegression(C=1.0, tol=1e-10, max_iter=10000)`, phạt L2, bộ giải lbfgs, huấn luyện trên cùng X_{train} đã chuẩn hoá. Đặt `tol` rất nhỏ để lbfgs chạy tới nghiệm chính xác; nhờ đó chênh lệch còn lại giữa hai bản, nếu có, đến từ thuật toán tối ưu chứ không do dừng sớm.

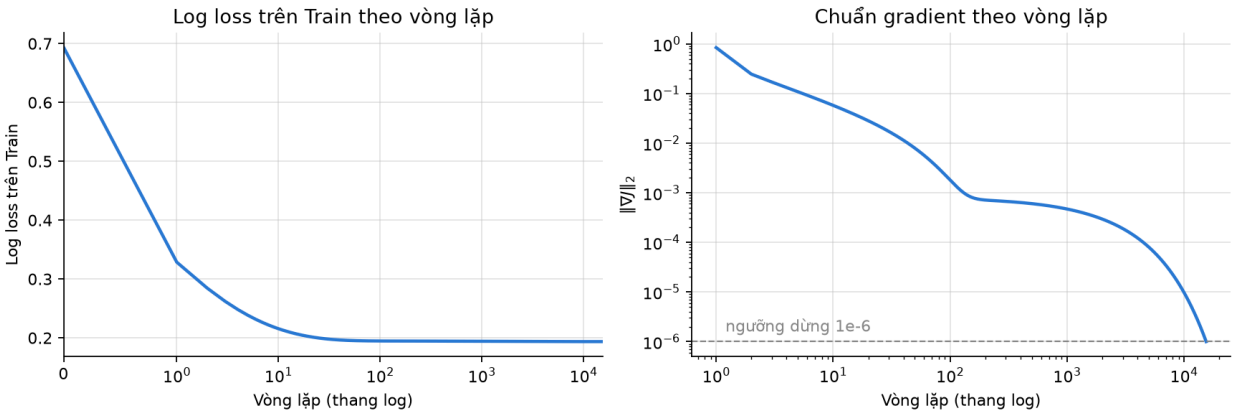
4.4. Baseline

`DummyClassifier(strategy="prior")`: luôn dự đoán xác suất bằng tỷ lệ lớp trên Train (Slide 6, tr. 8–9).

5. Thực nghiệm và kết quả

5.1. Quá trình huấn luyện

Gradient descent dừng theo điều kiện $\|\nabla J\| < 10^{-6}$ sau 15.476 vòng lặp. Log loss trên Train giảm từ 0,6931 xuống 0,1936: sau 10 vòng còn 0,2155, sau 100 vòng còn 0,1946, sau 1.000 vòng còn 0,1942.



Hình 2. Log loss trên Train và chuẩn gradient theo vòng lặp (bản NumPy).

5.2. Bảng so sánh trên tập Test

Ngưỡng quyết định τ = 0,5. ROC-AUC và log loss tính từ xác suất dự đoán; các chỉ số còn lại tính từ nhãn dự đoán.

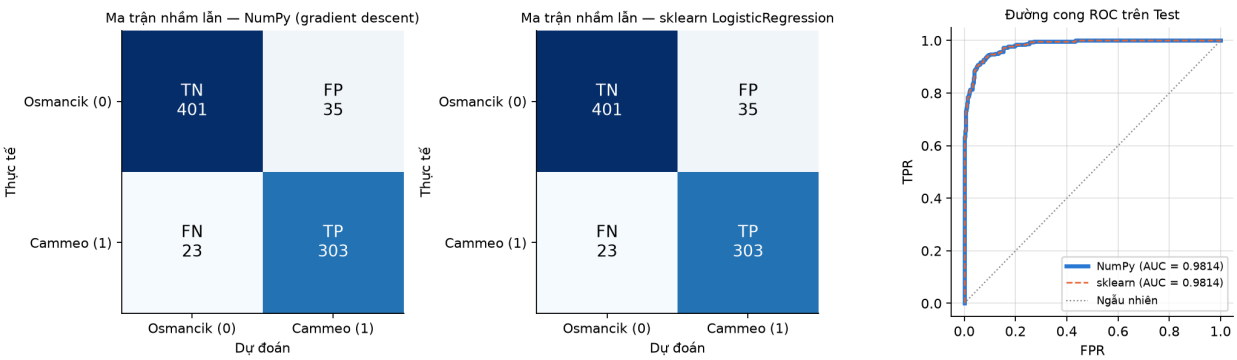
Mô hình	Accuracy	Precision	Recall	F1	ROC-AUC	Log loss
Baseline (theo tỷ lệ lớp)	0,5722	0,0000	0,0000	0,0000	0,5000	0,6827
NumPy, gradient descent	0,9239	0,8964	0,9294	0,9127	0,9814	0,1767
sklearn LogisticRegression	0,9239	0,8964	0,9294	0,9127	0,9814	0,1767

Bảng 2. Kết quả trên tập Test (762 mẫu).

5.3. Ma trận nhầm lẫn

Thực tế	NumPy — dự đoán		sklearn — dự đoán	
	Osmancik (0)	Cammeo (1)	Osmancik (0)	Cammeo (1)
Osmancik (0)	TN = 401	FP = 35	TN = 401	FP = 35
Cammeo (1)	FN = 23	TP = 303	FN = 23	TP = 303

Bảng 3. Ma trận nhầm lẫn của hai bản trên Test. Số mẫu hai bản dự đoán khác nhau: 0.



Hình 3. Ma trận nhầm lẫn của hai bản và đường cong ROC trên Test.

5.4. Hai bản có cho cùng một mô hình không?

Tham số	NumPy	sklearn	|chênh lệch|
Area	0,478278	0,477203	$1{,}1 \times 10^{-3}$
Perimeter	0,436460	0,436017	$4{,}4 \times 10^{-4}$
Major_Axis_Length	0,937057	0,937019	$3{,}8 \times 10^{-5}$
Minor_Axis_Length	-0,257752	-0,257997	$2{,}4 \times 10^{-4}$
Eccentricity	1,412588	1,412545	$4{,}3 \times 10^{-5}$
Convex_Area	2,028721	2,030460	$1{,}7 \times 10^{-3}$
Extent	0,045565	0,045558	$7{,}1 \times 10^{-6}$
bias b	-0,772030	-0,772011	$1{,}9 \times 10^{-5}$

Bảng 4. Trọng số (trên đặc trưng đã chuẩn hoá) và bias của hai bản.

Hàm mục tiêu J trên Train: NumPy 0,195243365, sklearn 0,195243364, chênh $1{,}1 \times 10^{-9}$. Xác suất dự đoán trên Test lệch tối đa $3{,}0 \times 10^{-5}$.

6. Nhận xét

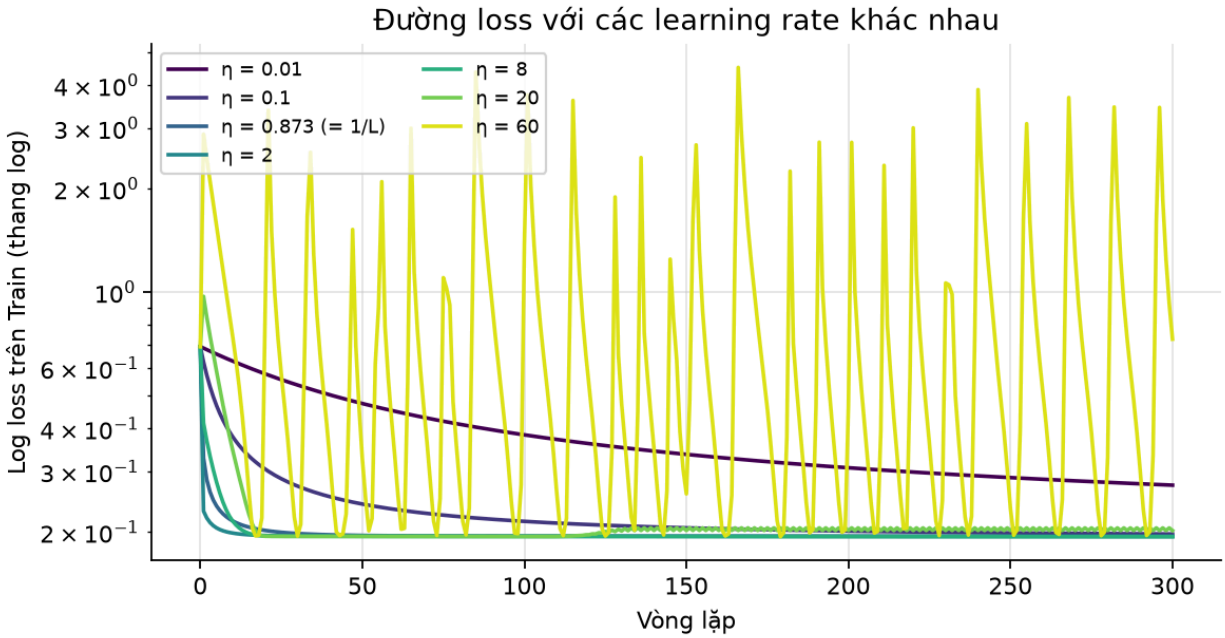
6.1. Sự hội tụ

Với $\eta = 1/L_{max}$, log loss giảm đơn điệu, không có vòng lặp nào loss tăng. Phần lớn mức giảm diễn ra trong 100 vòng đầu, nhưng phải tới vòng 15.476 chuẩn gradient mới xuống dưới 10^{-6}. Đuôi hội tụ dài này do cộng tuyến: Hessian tại nghiệm có trị riêng lớn nhất 0,106 và nhỏ nhất $4{,}6 \times 10^{-4}$, số điều kiện khoảng 231. Mặt loss có hướng rất phẳng, gradient descent đi rất chậm dọc hướng đó.

Thí nghiệm learning rate (300 vòng mỗi giá trị) cho đủ các dạng đường loss mô tả ở Slide 5, tr. 14 và tr. 18: $\eta = 0{,}01$ giảm quá chậm; η từ 0,87 đến 8 giảm đều; $\eta = 20$ dao động quanh nghiệm; $\eta = 60$ nhảy liên tục và không hội tụ. Độ cong thật lớn nhất (0,106) nhỏ hơn nhiều so với cận $L_{max} = 1{,}146$, nên $\eta = 1/L_{max}$ an toàn nhưng dè dặt, và ngưỡng mất ổn định thực tế nằm quanh $2/0{,}106 \approx 19$.

η	Log loss Train sau 300 vòng	Số vòng loss tăng
0,010	0,2739	0
0,100	0,1975	0
0,873	0,1945	0
2,000	0,1943	0
8,000	0,1939	0
20,000	0,2021	115
60,000	0,7288	80

Bảng 5. Ảnh hưởng của learning rate.



Hình 4. Đường loss với các learning rate khác nhau.

6.2. Chênh lệch giữa hai bản

- Vì tối ưu cùng một hàm mục tiêu, hai bản cho 762/762 dự đoán giống nhau trên Test, hai ma trận nhầm lẫn giống hệt, accuracy, precision, recall, F1 và ROC-AUC bằng nhau; log loss chỉ khác ở chữ số thập phân thứ sáu (0,176748 và 0,176747).
- Trọng số lệch nhiều hơn xác suất: tối đa $1{,}7 \times 10^{-3}$, ở Area và Convex_Area, hai đặc trưng tương quan 0,999. Dọc hướng phẳng của mặt loss, trọng số có thể dịch chuyển mà dự đoán hầu như không đổi. Mức lệch này khớp với ước lượng $\|\nabla J\| / \lambda_{min} \approx 10^{-6} / 4{,}6 \times 10^{-4} \approx 2{,}2 \times 10^{-3}$.
- Khác biệt thật sự là thuật toán: lbfgs dùng thông tin độ cong nên dừng sau 43 vòng; gradient descent chỉ dùng gradient nên cần 15.476 vòng.

- Nếu bản NumPy không có L2 trong khi sklearn dùng C = 1 mặc định, hai bản sẽ tối ưu hai hàm khác nhau và chênh lệch quan sát được sẽ là chênh lệch về mô hình chứ không phải về cài đặt.

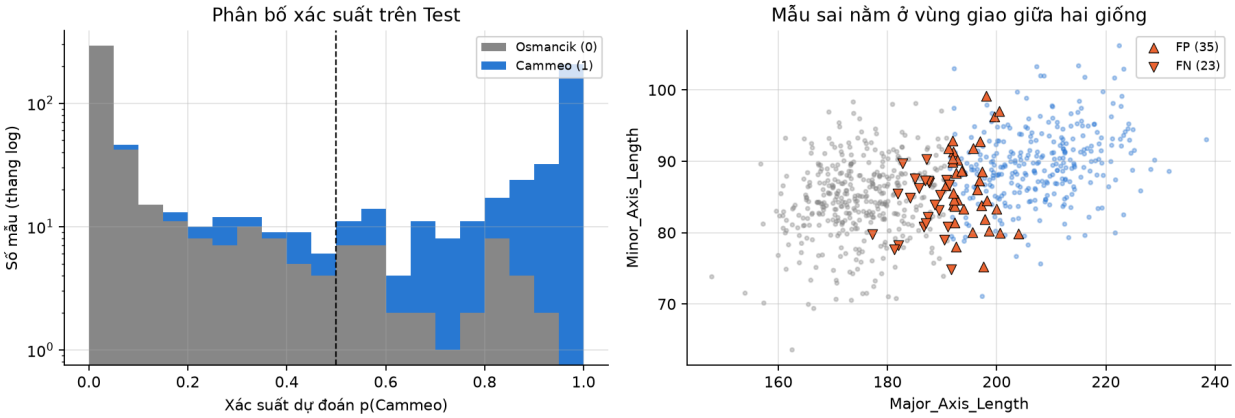
6.3. Các lỗi phân loại thường gặp

Mô hình sai 58/762 mẫu Test (7,6%): 35 FP (Osmancik bị đoán là Cammeo) và 23 FN (Cammeo bị đoán là Osmancik).

Nhóm	Số mẫu	Area	Trục chính	Trục phụ	p trung bình
TN — Osmancik đoán đúng	401	11.470	175,0	84,5	0,060
FP — Osmancik đoán thành Cammeo	35	13.120	195,2	86,6	0,698
FN — Cammeo đoán thành Osmancik	23	12.084	186,6	83,8	0,268
TP — Cammeo đoán đúng	303	14.430	207,7	89,5	0,931

Bảng 6. Giá trị trung bình của một số đặc trưng theo nhóm kết quả trên Test.

- **Sai ở đâu:** ở vùng giao giữa hai giống theo kích thước hạt. FP là các hạt Osmancik to và dài bất thường (Area trung bình 13.120, trong khi Osmancik đoán đúng là 11.470). FN là các hạt Cammeo nhỏ và ngắn (Area 12.084, so với 14.430 của Cammeo đoán đúng).
- **Mức độ tự tin:** 27/58 mẫu sai (47%) có xác suất trong khoảng (0,3; 0,7), tức nằm sát ngưỡng (Slide 6, tr. 49); chỉ 7 mẫu sai mà mô hình rất tự tin ($p < 0,1$ hoặc $p > 0,9$).
- **Vì sao:** nguyên nhân thuộc nhóm đặc trưng (nhóm F trong Slide 1, tr. 22) chứ không phải mô hình. Cả 7 đặc trưng đều mô tả kích thước và hình dạng, nên một hạt Osmancik to không phân biệt được với một hạt Cammeo nhỏ.
- **Sửa thế nào:** bổ sung đặc trưng khác loại (màu sắc, kết cấu bề mặt). Đổi thuật toán tối ưu không giúp gì, vì cả ba thuật toán đã thử đều tới cùng một nghiệm.

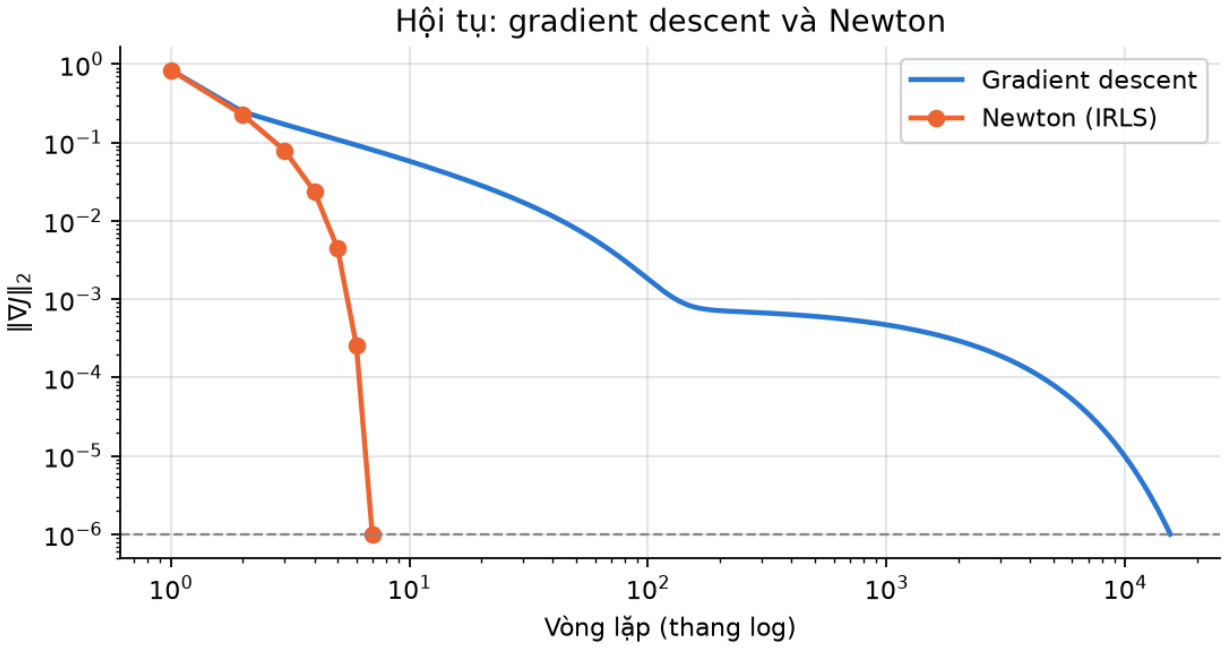


Hình 5. Phân bố xác suất dự đoán trên Test và vị trí các mẫu sai theo hai trục của hạt.

7. Phân tích mở rộng

7.1. Newton (IRLS) và vai trò của chuẩn hoá

Phương pháp Newton tự cài bằng NumPy, cập nhật $\theta \leftarrow \theta - H^{-1}\nabla J$, hội tụ sau 6 vòng với cùng ngưỡng dừng, vì không bị ảnh hưởng bởi số điều kiện. Ngược lại, khi **không** chuẩn hoá đặc trưng, L_{max} tăng lên $8{,}3 \times 10^{7}$ nên bước học an toàn chỉ còn $1{,}2 \times 10^{-8}$; sau 20.000 vòng log loss mới giảm tới 0,668 và accuracy Test là 0,671 (Slide 5, tr. 17).



Hình 6. Chuẩn gradient theo vòng lặp: gradient descent và Newton.

7.2. Batch, mini-batch và SGD

Phương pháp	η	Số bước cập nhật	Log loss Train sau 30 epoch
Batch (N mẫu)	0,873	30	0,1981
Mini-batch (B = 64)	0,300	1.071	0,1947
SGD (B = 1)	0,020	68.580	0,1942

Bảng 7. So sánh với cùng 30 lượt duyệt dữ liệu (Slide 5, tr. 16).

Tính theo số lượt duyệt dữ liệu, mini-batch và SGD xuống gần nghiệm nhanh hơn batch; đổi lại đường loss của SGD dao động quanh nghiệm thay vì giảm đều.

7.3. Chọn C bằng cross-validation

C	CV log loss (5 fold)	SE
0,001	0,3580	0,0023
0,01	0,2237	0,0021
0,1	0,1974	0,0016
1	0,1958	0,0017
10	0,1935	0,0017
100	0,1917	0,0020
1000	0,1917	0,0021

Bảng 8. Chọn C trên Train, scaler nằm trong pipeline và được fit lại ở mỗi fold (Slide 6, tr. 24 và tr. 45).

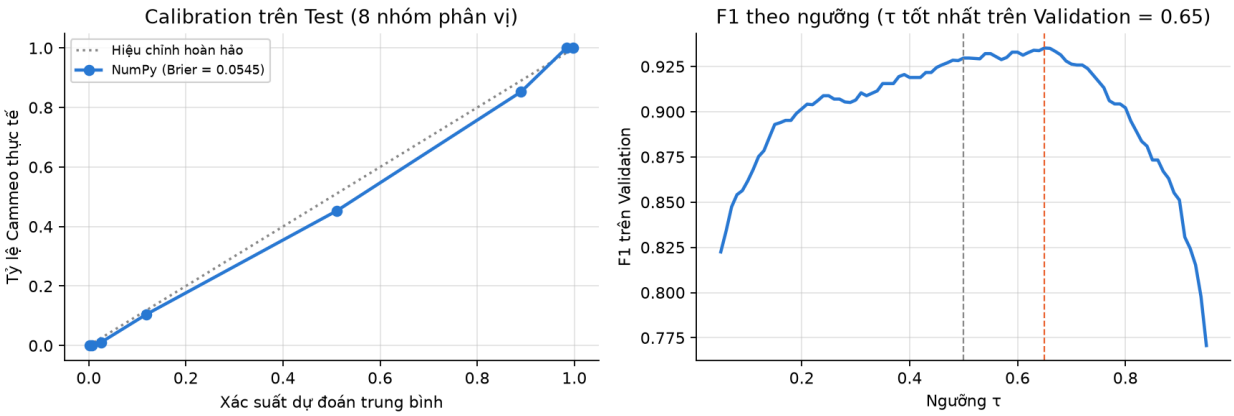
CV log loss thấp nhất ở C = 100; theo quy tắc một sai số chuẩn (Slide 5, tr. 45), C = 10 là mô hình đơn giản nhất còn trong ngưỡng. Trên Test, C = 100 cho accuracy 0,9304 và log loss 0,1700, so với 0,9239 và 0,1767 của C = 1. Mức cải thiện này nhỏ hơn bề rộng khoảng tin cậy ở Bảng 9.

7.4. Độ bất định, chất lượng xác suất và ngưỡng

Chỉ số	Giá trị trên Test	Khoảng tin cậy 95% (bootstrap)
Accuracy	0,9239	[0,904; 0,942]
Precision	0,8964	[0,861; 0,927]
Recall	0,9294	[0,899; 0,956]
F1	0,9127	[0,888; 0,934]
ROC-AUC	0,9814	[0,974; 0,988]
Log loss	0,1767	[0,148; 0,208]

Bảng 9. Khoảng tin cậy từ 2.000 lần bootstrap trên Test (bản NumPy).

- Hai bản không có cặp dự đoán bất đồng nào (kiểm định McNemar: 0 và 0), nên không có khác biệt để kiểm định.
- Brier score trên Test là 0,0545; đường calibration nằm gần đường chéo, lệch nhiều nhất ở nhóm xác suất quanh 0,5 (Slide 6, tr. 40–42).
- Quét ngưỡng trên Validation theo F1 cho $\tau = 0{,}65$. Áp dụng lên Test, F1 chỉ đổi từ 0,9127 lên 0,9134 (precision 0,939, recall 0,890), nên giữ $\tau = 0{,}5$ là hợp lý khi hai loại lỗi có chi phí như nhau (Slide 6, tr. 34–35).
- VIF của Area và Convex_Area trên 1.000, nên odds ratio của từng đặc trưng kích thước riêng lẻ không ổn định và không nên diễn giải tách rời (Slide 6, tr. 26–28).



Hình 7. Calibration trên Test và F1 theo ngưỡng trên Validation.

8. Kết luận

- Bản NumPy cài đúng: tái lập được ví dụ tính tay của slide, qua kiểm tra gradient, và cho cùng nghiệm với scikit-learn khi hai bản tối ưu cùng một hàm mục tiêu.
- Trên bộ Rice, Logistic Regression đạt accuracy 0,924 (khoảng tin cậy 95%: [0,904; 0,942]) và ROC-AUC 0,981, so với accuracy 0,572 của baseline.
- Gradient descent hội tụ chắc chắn với $\eta = 1/L_{max}$ nhưng chậm khi đặc trưng cộng tuyến; chuẩn hoá đặc trưng là điều kiện để nó chạy được trong thực tế.
- **Giới hạn:** mô hình chỉ dùng 7 đặc trưng hình thái nên không tách được các hạt có kích thước trung gian; kết quả áp dụng cho ảnh chụp trong cùng điều kiện với bộ dữ liệu gốc.

9. Mã nguồn và cách chạy

Mã nguồn công khai: https://github.com/RubyBarSchool/nhan-dang-mau-bt2-logistic-regression

```
pip install -r requirements.txt
jupyter nbconvert --to notebook --execute --inplace
B26CHKH096_TranVuTruong_BaiTap2.ipynb
```

Hoặc mở notebook và chọn *Restart & Run All*. Dữ liệu nằm sẵn trong `data/`, không cần tải thêm. Môi trường đã chạy: Python 3.12, NumPy 2.5.3, scikit-learn 1.9.1.

Tài liệu tham khảo

1. Nguyễn Xuân Đức. Bài giảng môn Nhận dạng mẫu, Học viện Công nghệ Bưu chính Viễn thông: Slide 1 “Tổng quan”; Slide 3 “Dữ liệu & đặc trưng”; Slide 5 “Mô hình hồi quy”; Slide 6 “Hồi quy logistic”. Số trang trích dẫn là số thứ tự trang trong file PDF.
2. C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. Mục 4.3 (Probabilistic Discriminative Models).

3. I. Cinar, M. Koklu. “Classification of Rice Varieties Using Artificial Intelligence Methods.” *International Journal of Intelligent Systems and Applications in Engineering*, 7(3), 2019, 188–194.
4. Rice (Cammeo and Osmancik). UCI Machine Learning Repository, 2019. https://archive.ics.uci.edu/dataset/545
5. F. Pedregosa và cộng sự. “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, 12, 2011, 2825–2830.